

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

О Т Ч Е Т

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
«Запросы на выборку и модификацию данных.
Представления. Работа с индексами»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Смирнов Фёдор Евгеньевич
Факультет прикладной информатики
Группа K3240
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии 2024
Преподаватель Говорова Марина Михайловна

Санкт-Петербург,
2026

Цель работы

Цель работы — овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема базы данных (ЛР3)

- БД: carsharing
- схема: rental
- вариант: 12
- предметная область: прокат автомобилей


```

SELECT
    COUNT(*) AS contracts_count,
    SUM(t.daily_rate * t.rental_days) AS rental_revenue,
    SUM(t.insurance_cost) AS insurance_revenue,
    SUM(t.daily_rate * t.rental_days + t.insurance_cost) AS total_revenue
FROM (
    SELECT
        rr.price AS daily_rate,
        COALESCE(rc.insurance_cost, 0) AS insurance_cost,
        (
            COALESCE(
                rc.return_datetime,
                (
                    SELECT MAX(ce.extension_end)
                    FROM rental.contract_extension ce
                    WHERE ce.contract_id = rc.id
                ),
                CURRENT_TIMESTAMP
            )::date
            - rc.issue_datetime::date
            + 1
        ) AS rental_days
    FROM rental.rental_contract rc
    JOIN rental.car c
        ON c.id = rc.car_id
    JOIN rental.rental_rate rr
        ON rr.model_id = c.model_id
        AND rr.start_date <= rc.issue_datetime::date
        AND (rr.end_date IS NULL OR rr.end_date >=
rc.issue_datetime::date)
    WHERE rc.issue_datetime::date >= (date_trunc('month', CURRENT_DATE) -
interval '1 month')::date
        AND rc.issue_datetime::date < date_trunc('month',
CURRENT_DATE)::date
    ) AS t;

```

Данный запрос предназначен для расчёта общей выручки компании от договоров проката за предыдущий календарный месяц. В запросе используются данные о договорах проката, тарифах аренды и стоимости страхования. Для каждого договора определяется стоимость аренды в зависимости от длительности проката и актуального тарифа, действовавшего на дату выдачи автомобиля. После этого вычисляется суммарная выручка от аренды, суммарная выручка от страхования и итоговое значение общей выручки.

```

1 SELECT
2     COUNT(*) AS contracts_count,
3     SUM(t.daily_rate * t.rental_days) AS rental_revenue,
4     SUM(t.insurance_cost) AS insurance_revenue,
5     SUM(t.daily_rate * t.rental_days + t.insurance_cost) AS total_revenue
6 FROM (
7     SELECT
8         rr.price AS daily_rate,
9         COALESCE(rc.insurance_cost, 0) AS insurance_cost,
10        (
11            COALESCE(
12                rc.return_datetime,
13                (
14                    SELECT MAX(ce.extension_end)
15                    FROM rental.contract_extension ce
16                    WHERE ce.contract_id = rc.id
17                ),
18                CURRENT_TIMESTAMP
19            )::date
20        - rc.issue_datetime::date
21        + 1
22        ) AS rental_days
23 FROM rental.rental_contract rc
24 JOIN rental.car c
25     ON c.id = rc.car_id
26 JOIN rental.rental_rate rr
27     ON rr.model_id = c.model_id
28     AND rr.start_date <= rc.issue_datetime::date
29     AND (rr.end_date IS NULL OR rr.end_date >= rc.issue_datetime::date)
30 WHERE rc.issue_datetime::date >= (date_trunc('month', CURRENT_DATE) - interval '1 month')::date
31     AND rc.issue_datetime::date < date_trunc('month', CURRENT_DATE)::date
32 ) AS t;

```

Data Output
Messages
Notifications

SQL

	contracts_count bigint	rental_revenue numeric	insurance_revenue numeric	total_revenue numeric
1	6	899300.00	15150.00	914450.00

Рисунок 2 — Результат выполнения запроса на расчёт выручки компании за последний календарный месяц.

1.2. Автомобили какой марки чаще всего брались в прокат SQL-команда.

```

SELECT
    m.brand,
    COUNT(*) AS rental_count
FROM rental.rental_contract rc
JOIN rental.car c
    ON c.id = rc.car_id
JOIN rental.model m
    ON m.id = c.model_id
GROUP BY m.brand
HAVING COUNT(*) = (
    SELECT MAX(brand_count)
    FROM (
        SELECT COUNT(*) AS brand_count
        FROM rental.rental_contract rc2
        JOIN rental.car c2
            ON c2.id = rc2.car_id
        JOIN rental.model m2
            ON m2.id = c2.model_id
    )
)

```

```
GROUP BY m2.brand  
) AS brand_counts  
);
```

Запрос определяет марку автомобиля, которая чаще всего встречается в заключённых договорах проката. Для этого используются таблицы договоров проката, автомобилей и моделей автомобилей. После соединения таблиц выполняется группировка по марке и подсчёт числа договоров, в которых фигурируют автомобили каждой марки. В результат выводится марка с максимальным количеством аренд.

```
1  SELECT  
2      m.brand,  
3      COUNT(*) AS rental_count  
4  FROM rental.rental_contract rc  
5  JOIN rental.car c  
6      ON c.id = rc.car_id  
7  JOIN rental.model m  
8      ON m.id = c.model_id  
9  GROUP BY m.brand  
10  HAVING COUNT(*) = (  
11      SELECT MAX(brand_count)  
12      FROM (  
13          SELECT COUNT(*) AS brand_count  
14          FROM rental.rental_contract rc2  
15          JOIN rental.car c2  
16              ON c2.id = rc2.car_id  
17          JOIN rental.model m2  
18              ON m2.id = c2.model_id  
19          GROUP BY m2.brand  
20      ) AS brand_counts  
21  );
```

Data Output Messages Notifications

	brand character varying (50)	rental_count bigint
1	Kia	8

Рисунок 3 — Результат выполнения запроса на определение марки автомобиля, наиболее часто использовавшейся в прокате.

1.3. Определить убытки от простоя автомобилей за вчерашний день SQL-команда.

```

SELECT
    COUNT(*) AS idle_cars_count,
    SUM(rr.price) AS idle_loss_yesterday
FROM rental.car c
JOIN rental.rental_rate rr
    ON rr.model_id = c.model_id
    AND rr.start_date <= (CURRENT_DATE - 1)
    AND (rr.end_date IS NULL OR rr.end_date >= (CURRENT_DATE - 1))
LEFT JOIN (
    SELECT DISTINCT cp.car_id
    FROM (
        SELECT
            rc.id,
            rc.car_id,
            rc.issue_datetime,
            COALESCE(rc.return_datetime, MAX(ce.extension_end)) AS
occupancy_end
        FROM rental.rental_contract rc
        LEFT JOIN rental.contract_extension ce
            ON ce.contract_id = rc.id
        GROUP BY
            rc.id,
            rc.car_id,
            rc.issue_datetime,
            rc.return_datetime
    ) AS cp
    WHERE cp.issue_datetime < CURRENT_DATE::timestamp
        AND COALESCE(cp.occupancy_end, CURRENT_DATE::timestamp) >
(CURRENT_DATE - 1)::timestamp
    ) AS bc
    ON bc.car_id = c.id
WHERE bc.car_id IS NULL;

```

Запрос позволяет определить количество автомобилей, которые не использовались в аренде в течение предыдущего дня, а также рассчитать возможную упущенную выручку от такого простоя. Для этого сначала определяются автомобили, занятые в прокате в рассматриваемый день, затем из общего списка автомобилей исключаются занятые машины. Для оставшихся автомобилей суммируется их суточный тариф аренды, что интерпретируется как условная величина потерь от простоя.

```

1  SELECT
2      COUNT(*) AS idle_cars_count,
3      SUM(rr.price) AS idle_loss_yesterday
4  FROM rental.car c
5  JOIN rental.rental_rate rr
6      ON rr.model_id = c.model_id
7      AND rr.start_date <= (CURRENT_DATE - 1)
8      AND (rr.end_date IS NULL OR rr.end_date >= (CURRENT_DATE - 1))
9  LEFT JOIN (
10     SELECT DISTINCT cp.car_id
11     FROM (
12         SELECT
13             rc.id,
14             rc.car_id,
15             rc.issue_datetime,
16             COALESCE(rc.return_datetime, MAX(ce.extension_end)) AS occupancy_end
17         FROM rental.rental_contract rc
18         LEFT JOIN rental.contract_extension ce
19             ON ce.contract_id = rc.id
20         GROUP BY
21             rc.id,
22             rc.car_id,
23             rc.issue_datetime,
24             rc.return_datetime
25     ) AS cp
26     WHERE cp.issue_datetime < CURRENT_DATE::timestamp
27     AND COALESCE(cp.occupancy_end, CURRENT_DATE::timestamp) > (CURRENT_DATE - 1)::timestamp
28 ) AS bc
29     ON bc.car_id = c.id
30 WHERE bc.car_id IS NULL;

```

Data Output Messages Notifications

	idle_cars_count bigint	idle_loss_yesterday numeric
1	5	19350.00

Рисунок 4 — Результат выполнения запроса на определение убытков от простоя автомобилей за предыдущий день.

1.4. Вывести данные автомобиля, имеющего максимальный пробег

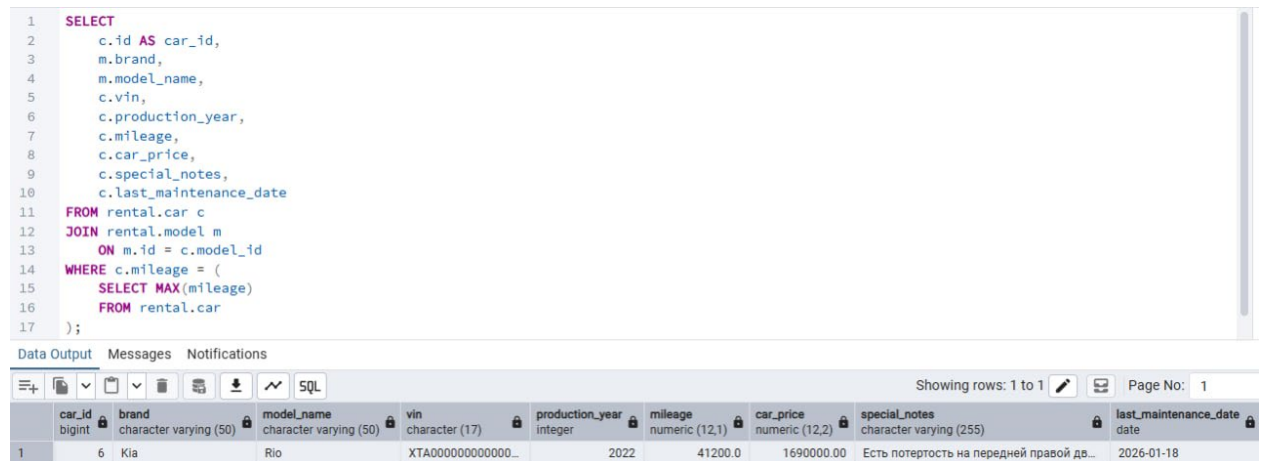
SQL-команда.

```

SELECT
    c.id AS car_id,
    m.brand,
    m.model_name,
    c.vin,
    c.production_year,
    c.mileage,
    c.car_price,
    c.special_notes,
    c.last_maintenance_date
FROM rental.car c
JOIN rental.model m
    ON m.id = c.model_id
WHERE c.mileage = (
    SELECT MAX(mileage)
    FROM rental.car
);

```


Данный запрос находит автомобиль с наибольшим пробегом среди всех автомобилей, зарегистрированных в базе данных. Сначала определяется максимальное значение пробега, после чего выводятся сведения об автомобиле или автомобилях, которым соответствует найденное максимальное значение. В результате можно определить наиболее интенсивно использовавшийся автомобиль компании.



```
1 SELECT
2   c.id AS car_id,
3   m.brand,
4   m.model_name,
5   c.vin,
6   c.production_year,
7   c.mileage,
8   c.car_price,
9   c.special_notes,
10  c.last_maintenance_date
11 FROM rental.car c
12 JOIN rental.model m
13   ON m.id = c.model_id
14 WHERE c.mileage = (
15   SELECT MAX(mileage)
16   FROM rental.car
17 );
```

car_id	brand	model_name	vin	production_year	mileage	car_price	special_notes	last_maintenance_date
6	Kia	Rio	XTA0000000000000...	2022	41200.0	1690000.00	Есть потертость на передней правой дв...	2026-01-18

Рисунок 5 — Результат выполнения запроса на определение автомобиля с максимальным пробегом.

1.5. Какой автомобиль суммарно находился в прокате дольше всех SQL-команда.

```

SELECT
    t.car_id,
    t.total_rental_days
FROM (
    SELECT
        rc.car_id,
        SUM(
            COALESCE(
                rc.return_datetime,
                (
                    SELECT MAX(ce.extension_end)
                    FROM rental.contract_extension ce
                    WHERE ce.contract_id = rc.id
                ),
                CURRENT_TIMESTAMP
            )::date
            - rc.issue_datetime::date
            + 1
        ) AS total_rental_days
    FROM rental.rental_contract rc
    GROUP BY rc.car_id
) AS t
WHERE t.total_rental_days = (
    SELECT MAX(x.total_rental_days)
    FROM (
        SELECT
            SUM(
                COALESCE(
                    rc2.return_datetime,
                    (
                        SELECT MAX(ce2.extension_end)
                        FROM rental.contract_extension ce2
                        WHERE ce2.contract_id = rc2.id
                    ),
                    CURRENT_TIMESTAMP
                )::date
                - rc2.issue_datetime::date
                + 1
            ) AS total_rental_days
        FROM rental.rental_contract rc2
        GROUP BY rc2.car_id
    ) AS x
);

```

Запрос определяет автомобиль, который суммарно провёл в аренде наибольшее количество времени. Для этого по каждому автомобилю суммируется длительность всех договоров проката, а при наличии продлений учитываются дополнительные периоды использования автомобиля. Далее выполняется сравнение суммарных длительностей по всем автомобилям, и в результат выводится автомобиль-лидер.

```

1  SELECT t.car_id, t.total_rental_days
2  FROM (
3      SELECT rc.car_id,
4             SUM(COALESCE(
5                 rc.return_datetime,
6                 (SELECT MAX(ce.extension_end)
7                   FROM rental.contract_extension ce
8                   WHERE ce.contract_id = rc.id),
9                 CURRENT_TIMESTAMP
10             )::date - rc.issue_datetime::date + 1) AS total_rental_days
11      FROM rental.rental_contract rc
12      GROUP BY rc.car_id
13  ) AS t
14  WHERE t.total_rental_days = (
15      SELECT MAX(x.total_rental_days)
16      FROM (
17          SELECT SUM(COALESCE(
18              rc2.return_datetime,
19              (SELECT MAX(ce2.extension_end)
20                FROM rental.contract_extension ce2
21                WHERE ce2.contract_id = rc2.id),
22              CURRENT_TIMESTAMP
23          )::date - rc2.issue_datetime::date + 1) AS total_rental_days
24          FROM rental.rental_contract rc2
25          GROUP BY rc2.car_id
26      ) AS x
27  );

```

Data Output Messages Notifications



	car_id bigint	total_rental_days bigint
1	3	282

Рисунок 6 — Результат выполнения запроса на определение автомобиля, суммарно дольше всех находившегося в прокате.

1.6. Определить, каким количеством автомобилей каждой марки и модели владеет компания

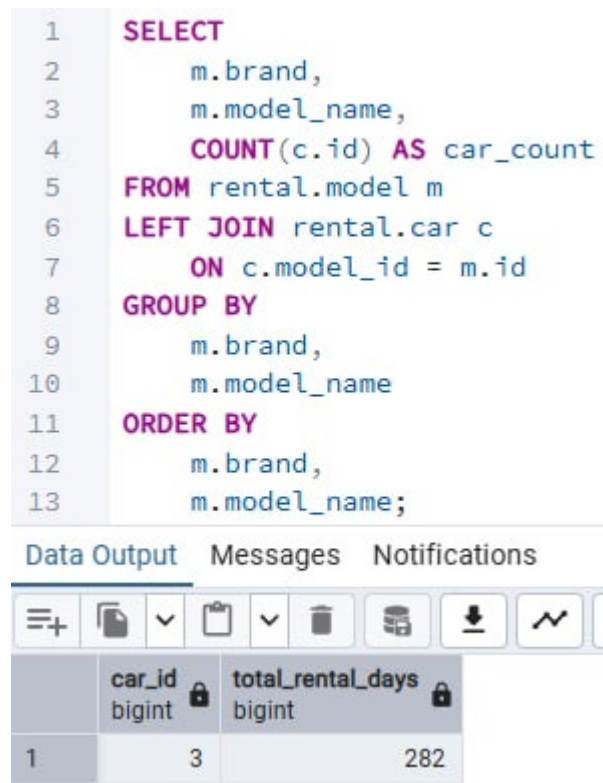
SQL-команда.

```

SELECT
    m.brand,
    m.model_name,
    COUNT(c.id) AS car_count
FROM rental.model m
LEFT JOIN rental.car c
    ON c.model_id = m.id
GROUP BY
    m.brand,
    m.model_name
ORDER BY
    m.brand,
    m.model_name;

```

Запрос формирует сводную информацию по автопарку компании. На основе таблиц моделей и конкретных автомобилей производится группировка по марке и модели автомобиля, после чего для каждой группы подсчитывается количество автомобилей. Полученный результат позволяет оценить состав автопарка и распределение машин по моделям.



```
1 SELECT
2     m.brand,
3     m.model_name,
4     COUNT(c.id) AS car_count
5 FROM rental.model m
6 LEFT JOIN rental.car c
7     ON c.model_id = m.id
8 GROUP BY
9     m.brand,
10    m.model_name
11 ORDER BY
12     m.brand,
13     m.model_name;
```

Data Output Messages Notifications

	car_id bigint	total_rental_days bigint
1	3	282

Рисунок 7 — Результат выполнения запроса на определение количества автомобилей по каждой марке и модели.

1.7. Определить средний возраст автомобилей компании

SQL-команда.

```
SELECT ROUND(AVG(t.service_life_years), 2) AS avg_car_age_years
FROM (
    SELECT
        (CURRENT_DATE - TO_DATE(c.production_year::text || '-01-01',
        'YYYY-MM-DD')) / 365.25 AS service_life_years
    FROM rental.car c
) AS t;
```

Данный запрос вычисляет средний возраст автомобилей компании по году выпуска. Для каждого автомобиля условно формируется дата производства как 1 января соответствующего года выпуска. После этого определяется разница между текущей датой и датой производства, а затем рассчитывается среднее значение по всем автомобилям базы данных. Такой показатель позволяет оценить общий уровень обновлённости автопарка компании.

1	SELECT ROUND(AVG(t.service_life_years), 2) AS avg_car_age_years
2	FROM (
3	SELECT
4	(CURRENT_DATE - TO_DATE(c.production_year::text '-01-01', 'YYYY-MM-DD')) / 365.25 AS service_life_years
5	FROM rental.car c
6	) AS t;

Data Output	Messages	Notifications
-------------	----------	---------------

+	SQL	Showing row
---	-----	-------------

avg_car_age_years	numeric
1	3.18

Рисунок 8 — Результат выполнения запроса на определение среднего возраста автомобилей компании.

2. Представления

2.1. Какой автомобиль ни разу не был в прокате

SQL-команда создания представления.

```
CREATE OR REPLACE VIEW rental.v_never_rented_cars AS
SELECT
  c.id AS car_id,
  m.brand,
  m.model_name,
  c.vin,
  c.production_year,
  c.mileage,
  c.car_price,
  c.special_notes,
  c.last_maintenance_date
FROM rental.car c
JOIN rental.model m
  ON m.id = c.model_id
LEFT JOIN rental.rental_contract rc
  ON rc.car_id = c.id
WHERE rc.id IS NULL;
```

Данное представление предназначено для отображения автомобилей, которые присутствуют в автопарке компании, но ни разу не фигурировали в договорах проката. Для его формирования используется соединение таблиц автомобилей, моделей и договоров проката. В итоговый набор попадают только те автомобили, для которых в таблице договоров отсутствуют связанные записи. Представление удобно использовать для анализа неиспользуемых автомобилей.

1	SELECT * FROM rental.v_never_rented_cars;
---	---

Data Output	Messages	Notifications
-------------	----------	---------------

+	SQL	Showing rows: 1 to 2 of
---	-----	-------------------------

car_id	brand	model_name	vin	production_year	mileage	car_price	special_notes	last_maintenance_date
10	Renault	Duster	ХТА0000000000000000...	2024	15400.0	2350000.00	Автомобиль в резерве, пока без договоров	2026-03-15
9	Lada	Vesta	ХТА0000000000000000...	2025	1800.0	1540000.00	Новый автомобиль, в аренду пока не выдава...	2026-03-28

Рисунок 9 — Содержимое представления v_never_rented_cars.

2.2. Вывести данные клиентов, не вернувших автомобиль вовремя

SQL-команда создания представления.

```
CREATE OR REPLACE VIEW rental.v_overdue_clients AS
SELECT
    cl.id AS client_id,
    cl.full_name,
    cl.phone,
    rc.id AS contract_id,
    rc.contract_number,
    le.agreed_return_datetime,
    rc.contract_status
FROM rental.rental_contract rc
JOIN rental.passport p
    ON p.id = rc.passport_id
JOIN rental.client cl
    ON cl.id = p.client_id
LEFT JOIN (
    SELECT
        ce.contract_id,
        MAX(ce.extension_end) AS agreed_return_datetime
    FROM rental.contract_extension ce
    GROUP BY ce.contract_id
) AS le
    ON le.contract_id = rc.id
WHERE rc.return_datetime IS NULL
AND (
    rc.contract_status = 'просрочен'
    OR (
        le.agreed_return_datetime IS NOT NULL
        AND le.agreed_return_datetime < CURRENT_TIMESTAMP
    )
);
```

Данное представление предназначено для выявления клиентов, не вернувших автомобиль в установленный срок. При формировании результата используются сведения о договоре проката, клиенте и продлениях договора. В итоговый набор попадают клиенты, у которых срок возврата автомобиля уже истёк, а сам договор остаётся открытым либо имеет статус просроченного. Представление позволяет быстро получить список проблемных договоров.

1 **SELECT** * **FROM** rental.v_overdue_clients;

Data Output Messages Notifications

Showing rows

	client_id bigint	full_name character varying (100)	phone character varying (20)	contract_id bigint	contract_number character varying (20)	agreed_return_datetime timestamp without time zone	contract_status character varying (20)
1	3	Смирнов Дмитрий Олегов...	+7-931-333-33-33	3	RC-2025-003	2025-05-25 11:00:00	открыт
2	5	Воробьев Кирилл Андрее...	+7-911-777-77-02	25	RC-2026-025	2026-04-02 12:00:00	просрочен
3	8	Захарова Елена Викторов...	+7-911-777-77-05	28	RC-2026-028	2026-04-06 14:00:00	просрочен

Рисунок 10 — Содержимое представления v_overdue_clients.

3. Модификация данных

3.1. Запрос INSERT с подзапросом

Формулировка запроса.

Добавить новое тестовое продление для последнего открытого договора проката.

SQL-команда.

```
BEGIN;

SELECT *
FROM rental.contract_extension
ORDER BY id;

INSERT INTO rental.contract_extension (
    contract_id,
    extension_start,
    extension_end,
    reason
)
SELECT
    t.contract_id,
    t.extension_start,
    t.extension_start + INTERVAL '2 days' AS extension_end,
    'Тестовое продление для ЛР4'
FROM (
    SELECT
        rc.id AS contract_id,
        COALESCE(
            (
                SELECT MAX(ce.extension_end)
                FROM rental.contract_extension ce
                WHERE ce.contract_id = rc.id
            ),
            rc.issue_datetime
        ) AS extension_start
    FROM rental.rental_contract rc
    WHERE rc.id = (
        SELECT MAX(rc2.id)
        FROM rental.rental_contract rc2
        WHERE rc2.contract_status = 'открыт'
    )
) AS t;

SELECT *
FROM rental.contract_extension
ORDER BY id;

ROLLBACK;
```

В данном примере реализована вставка новой записи в таблицу `contract_extension`. Подзапрос используется для определения последнего открытого договора, а также для вычисления даты окончания последнего продления по выбранному договору. Таким образом, новая запись продления формируется не вручную, а на основе уже имеющихся данных в базе. До выполнения запроса и после него было показано содержимое таблицы, что позволило сравнить результат вставки.


```

1 SELECT *
2 FROM rental.contract_extension
3 ORDER BY id;

```

	id [PK] bigint	contract_id bigint	extension_start timestamp without time zone	extension_end timestamp without time zone	reason character varying (255)
1	1	3	2025-05-23 11:00:00	2025-05-25 11:00:00	Клиент запросил продление поездки
2	2	24	2026-03-31 10:00:00	2026-12-15 10:00:00	Клиент продлил аренду до конца командировки
3	3	25	2026-04-01 12:00:00	2026-04-02 12:00:00	Клиент запросил краткосрочное продление, срок уже ис...
4	4	26	2026-04-04 09:00:00	2026-12-20 09:00:00	Продление для длительной аренды
5	5	28	2026-04-05 14:00:00	2026-04-06 14:00:00	Просроченное продление без последующего возврата
6	6	29	2026-04-09 09:00:00	2026-12-10 09:00:00	Продление корпоративной аренды

Рисунок 11 — Состояние таблицы `contract_extension` до выполнения запроса `INSERT`.

```

1 SELECT *
2 FROM rental.contract_extension
3 ORDER BY id;

```

	id [PK] bigint	contract_id bigint	extension_start timestamp without time zone	extension_end timestamp without time zone	reason character varying (255)
1	1	3	2025-05-23 11:00:00	2025-05-25 11:00:00	Клиент запросил продление поездки
2	2	24	2026-03-31 10:00:00	2026-12-15 10:00:00	Клиент продлил аренду до конца командировки
3	3	25	2026-04-01 12:00:00	2026-04-02 12:00:00	Клиент запросил краткосрочное продление, срок уже ис...
4	4	26	2026-04-04 09:00:00	2026-12-20 09:00:00	Продление для длительной аренды
5	5	28	2026-04-05 14:00:00	2026-04-06 14:00:00	Просроченное продление без последующего возврата
6	6	29	2026-04-09 09:00:00	2026-12-10 09:00:00	Продление корпоративной аренды
7	8	29	2026-12-10 09:00:00	2026-12-12 09:00:00	Тестовое продление для ЛР4

Рисунок 12 — Состояние таблицы `contract_extension` после выполнения запроса `INSERT`.

3.2. Запрос UPDATE с подзапросом

Формулировка запроса.

Изменить данные клиента, который чаще всего пользовался прокатом, увеличив его скидку и установив признак постоянного клиента.

SQL-команда.

```

BEGIN;

SELECT
    c.id,
    c.full_name,
    c.regular_customer,
    c.discount_percent
FROM rental.client c
ORDER BY c.id;

UPDATE rental.client
SET
    regular_customer = 'Y',
    discount_percent = CASE
        WHEN discount_percent + 5 > 20 THEN 20
        ELSE discount_percent + 5
    END
WHERE id IN (
    SELECT p.client_id

```



```

FROM rental.passport p
JOIN rental.rental_contract rc
    ON rc.passport_id = p.id
GROUP BY p.client_id
HAVING COUNT(*) = (
    SELECT MAX(client_contract_count)
    FROM (
        SELECT COUNT(*) AS client_contract_count
        FROM rental.passport p2
        JOIN rental.rental_contract rc2
            ON rc2.passport_id = p2.id
        GROUP BY p2.client_id
    ) AS client_counts
)
);

SELECT
    c.id,
    c.full_name,
    c.regular_customer,
    c.discount_percent
FROM rental.client c
ORDER BY c.id;

ROLLBACK;

```

Запрос обновляет запись клиента, выбранного с помощью подзапроса. Подзапрос определяет клиента с максимальным числом договоров проката, после чего основной запрос изменяет значение скидки и признак постоянного клиента. Для наглядности до и после выполнения UPDATE был выполнен просмотр таблицы клиентов.

```

1  SELECT
2      c.id,
3      c.full_name,
4      c.regular_customer,
5      c.discount_percent
6  FROM rental.client c
7  ORDER BY c.id;

```

Data Output Messages Notifications

	id [PK] bigint	full_name character varying (100)	regular_customer character (1)	discount_percent numeric (5,2)
1	1	Иванов Иван Сергеевич	Y	10.00
2	2	Петрова Анна Викторовна	N	0.00
3	3	Смирнов Дмитрий Олегов...	Y	5.00
4	4	Алексеева Марина Игор...	N	0.00
5	5	Воробьев Кирилл Андрее...	Y	7.50
6	6	Киселева Ольга Романов...	N	0.00
7	7	Морозов Артем Павлович	Y	12.00
8	8	Захарова Елена Викторов...	N	3.00

Рисунок 13 — Состояние таблицы client до выполнения запроса UPDATE.

```

1  SELECT
2      c.id,
3      c.full_name,
4      c.regular_customer,
5      c.discount_percent
6  FROM rental.client c
7  ORDER BY c.id;

```

Data Output Messages Notifications

	id [PK] bigint	full_name character varying (100)	regular_customer character (1)	discount_percent numeric (5,2)
1	1	Иванов Иван Сергеевич	Y	15.00
2	2	Петрова Анна Викторовна	Y	5.00
3	3	Смирнов Дмитрий Олегов...	Y	5.00
4	4	Алексеева Марина Игор...	Y	5.00
5	5	Воробьев Кирилл Андрее...	Y	12.50
6	6	Киселева Ольга Романов...	Y	5.00
7	7	Морозов Артем Павлович	Y	17.00
8	8	Захарова Елена Викторов...	Y	8.00

Рисунок 14 — Состояние таблицы client после выполнения запроса UPDATE.

Может показаться, что обновляются все клиенты, но дело в том, что у всех остальных кроме Смирнова по 4 договора, а у него всего 2.

3.3. Запрос DELETE с подзапросом

Формулировка запроса.

Удалить последнее продление у последнего открытого договора.

SQL-команда.

```
BEGIN;

SELECT *
FROM rental.contract_extension
ORDER BY id;

DELETE FROM rental.contract_extension
WHERE id = (
    SELECT MAX(ce.id)
    FROM rental.contract_extension ce
    WHERE ce.contract_id = (
        SELECT MAX(rc.id)
        FROM rental.rental_contract rc
        WHERE rc.contract_status = 'открыт'
    )
    AND ce.extension_end = (
        SELECT MAX(ce2.extension_end)
        FROM rental.contract_extension ce2
        WHERE ce2.contract_id = ce.contract_id
    )
);

SELECT *
FROM rental.contract_extension
ORDER BY id;

ROLLBACK;
```

В этом примере выполняется удаление записи о продлении договора. Подзапрос определяет последнее продление для последнего открытого договора, после чего основной запрос удаляет только найденную строку. Сравнение состояния таблицы до и после выполнения команды позволяет продемонстрировать корректность выполнения DELETE.

1

2

3

SELECT *

FROM rental.contract_extension

ORDER BY id;

Data Output

Messages

Notifications

	id [PK] bigint	contract_id bigint	extension_start timestamp without time zone	extension_end timestamp without time zone	reason character varying (255)
1	1	3	2025-05-23 11:00:00	2025-05-25 11:00:00	Клиент запросил продление поездки
2	2	24	2026-03-31 10:00:00	2026-12-15 10:00:00	Клиент продлил аренду до конца командировки
3	3	25	2026-04-01 12:00:00	2026-04-02 12:00:00	Клиент запросил краткосрочное продление, срок уже ис...
4	4	26	2026-04-04 09:00:00	2026-12-20 09:00:00	Продление для длительной аренды
5	5	28	2026-04-05 14:00:00	2026-04-06 14:00:00	Просроченное продление без последующего возврата
6	6	29	2026-04-09 09:00:00	2026-12-10 09:00:00	Продление корпоративной аренды

Рисунок 15 — Состояние таблицы contract_extension до выполнения запроса DELETE.

```

1 SELECT *
2 FROM rental.contract_extension
3 ORDER BY id;

```

	id [PK] bigint	contract_id bigint	extension_start timestamp without time zone	extension_end timestamp without time zone	reason character varying (255)
1	1	3	2025-05-23 11:00:00	2025-05-25 11:00:00	Клиент запросил продление поездки
2	2	24	2026-03-31 10:00:00	2026-12-15 10:00:00	Клиент продлил аренду до конца командировки
3	3	25	2026-04-01 12:00:00	2026-04-02 12:00:00	Клиент запросил краткосрочное продление, срок уже ис...
4	4	26	2026-04-04 09:00:00	2026-12-20 09:00:00	Продление для длительной аренды
5	5	28	2026-04-05 14:00:00	2026-04-06 14:00:00	Просроченное продление без последующего возврата

Рисунок 16 — Состояние таблицы `contract_extension` после выполнения запроса `DELETE`.

4. Индексы и планы выполнения

В заключительной части лабораторной работы был выполнен анализ двух запросов с использованием индексов и команды `EXPLAIN ANALYZE`. Согласно методичке, необходимо было выполнить запросы без индексов, получить планы выполнения, затем создать простой и составной индексы, повторно выполнить те же запросы, сравнить время выполнения и удалить созданные индексы. Для просмотра результатов использовались вкладки Data Output, Messages, Query History и Explain в Query Tool.

4.1. Эксперимент с простым индексом

Назначение эксперимента.

Проверить влияние простого индекса по полю `contract_status` на выполнение запроса отбора открытых договоров.

SQL-команды.

```

DROP INDEX IF EXISTS rental.idx_rental_contract_status;

EXPLAIN ANALYZE
SELECT *
FROM rental.rental_contract
WHERE contract_status = 'открыт';

CREATE INDEX idx_rental_contract_status
ON rental.rental_contract (contract_status);

EXPLAIN ANALYZE
SELECT *
FROM rental.rental_contract
WHERE contract_status = 'открыт';

DROP INDEX IF EXISTS rental.idx_rental_contract_status;

```

Для первого эксперимента был выбран запрос к таблице `rental_contract` с фильтрацией по значению поля `contract_status`. До создания индекса был получен план выполнения запроса с помощью `EXPLAIN ANALYZE`. После этого был создан простой индекс по полю `contract_status`, и запрос был выполнен повторно. Сравнение планов и времени выполнения показало, что на малом объёме тестовых данных планировщик PostgreSQL сохраняет последовательное чтение таблицы (Seq Scan), поскольку стоимость полного

просмотра небольшой таблицы оказывается ниже стоимости использования индекса.

1	EXPLAIN ANALYZE
2	SELECT *
3	FROM rental.rental_contract
4	WHERE contract_status = 'открыт';

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗑

📄

📄

📄

SQL

	QUERY PLAN	🔒
	text	
1	Seq Scan on rental_contract (cost=0.00..1.38 rows=1 width=350) (actual time=0.011..0.015 rows=4 loops=1)	
2	Filter: ((contract_status)::text = 'открыт'::text)	
3	Rows Removed by Filter: 26	
4	Planning Time: 0.106 ms	
5	Execution Time: 0.176 ms	

Рисунок 17 — План выполнения запроса по полю `contract_status` без индекса.

1	EXPLAIN ANALYZE
2	SELECT *
3	FROM rental.rental_contract
4	WHERE contract_status = 'открыт';

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗑

📄

📄

📄

SQL

	QUERY PLAN	🔒
	text	
1	Seq Scan on rental_contract (cost=0.00..1.38 rows=1 width=350) (actual time=0.011..0.015 rows=4 loops=1)	
2	Filter: ((contract_status)::text = 'открыт'::text)	
3	Rows Removed by Filter: 26	
4	Planning Time: 0.347 ms	
5	Execution Time: 0.030 ms	

Рисунок 18 — План выполнения запроса по полю `contract_status` после создания простого индекса.

До создания индекса для запроса были получены значения Planning Time = 0.106 мс и Execution Time = 0.176 мс. После создания простого индекса по полю `contract_status` были получены значения Planning Time = 0.347 мс и Execution Time = 0.030 мс. Если рассматривать суммарное наблюдаемое время работы команды EXPLAIN ANALYZE, то оно составило 0.282 мс до создания индекса и 0.377 мс после создания индекса. Несмотря на наличие индекса, при небольшом объёме таблицы `rental_contract` планировщик PostgreSQL может сохранять последовательное чтение таблицы (Seq Scan), поскольку стоимость полного просмотра таблицы остаётся низкой. При этом фактическое время выполнения самого запроса после создания индекса оказалось меньше.

4.2. Эксперимент с составным индексом

Назначение эксперимента.

Проверить влияние составного индекса по полям `model_id` и `start_date` на выполнение запроса выборки тарифов аренды.

SQL-команды.

```
DROP INDEX IF EXISTS rental.idx_rental_rate_model_start;

EXPLAIN ANALYZE
SELECT *
FROM rental.rental_rate
WHERE model_id = 3
      AND start_date <= DATE '2025-05-20'
ORDER BY start_date DESC;

CREATE INDEX idx_rental_rate_model_start
ON rental.rental_rate (model_id, start_date DESC);

EXPLAIN ANALYZE
SELECT *
FROM rental.rental_rate
WHERE model_id = 3
      AND start_date <= DATE '2025-05-20'
ORDER BY start_date DESC;

DROP INDEX IF EXISTS rental.idx_rental_rate_model_start;
```

Для второго эксперимента был выбран запрос к таблице `rental_rate`, в котором производится отбор записей по модели автомобиля и дате начала действия тарифа, а затем выполняется сортировка по дате. Для такого запроса был создан составной индекс по полям `model_id` и `start_date`. После повторного выполнения `EXPLAIN ANALYZE` было проведено сравнение плана выполнения и времени работы запроса. На тестовом наборе малого объёма PostgreSQL также выбрал последовательное чтение таблицы с последующей сортировкой, что является допустимым результатом при небольшом числе строк.

1	EXPLAIN ANALYZE
2	SELECT *
3	FROM rental.rental_rate
4	WHERE model_id = 3
5	AND start_date <= DATE '2025-05-20'
6	ORDER BY start_date DESC;

Data Output	Messages	Notifications
-------------	----------	---------------

≡+	📄	▼	📋	▼	🗑️	🗄️	⬇️	📈	SQL
----	---	---	---	---	----	----	----	---	-----

	QUERY PLAN	🔒
	text	
1	Sort (cost=1.22..1.22 rows=1 width=40) (actual time=0.448..0.448 rows=1 loops=1)	
2	Sort Key: start_date DESC	
3	Sort Method: quicksort Memory: 25kB	
4	-> Seq Scan on rental_rate (cost=0.00..1.21 rows=1 width=40) (actual time=0.026..0.027 rows=1 loops=1)	
5	Filter: ((start_date <= '2025-05-20'::date) AND (model_id = 3))	
6	Rows Removed by Filter: 13	
7	Planning Time: 3.787 ms	
8	Execution Time: 0.462 ms	

Рисунок 19 — План выполнения запроса к таблице `rental_rate` без составного индекса.

1	EXPLAIN ANALYZE
2	SELECT *
3	FROM rental.rental_rate
4	WHERE model_id = 3
5	AND start_date <= DATE '2025-05-20'
6	ORDER BY start_date DESC;

Data Output	Messages	Notifications
-------------	----------	---------------

≡+	📄	▼	📋	▼	🗑️	🗄️	⬇️	📈	SQL
----	---	---	---	---	----	----	----	---	-----

	QUERY PLAN	🔒
	text	
1	Sort (cost=1.22..1.22 rows=1 width=40) (actual time=0.042..0.042 rows=1 loops=1)	
2	Sort Key: start_date DESC	
3	Sort Method: quicksort Memory: 25kB	
4	-> Seq Scan on rental_rate (cost=0.00..1.21 rows=1 width=40) (actual time=0.036..0.037 rows=1 loops=1)	
5	Filter: ((start_date <= '2025-05-20'::date) AND (model_id = 3))	
6	Rows Removed by Filter: 13	
7	Planning Time: 0.373 ms	
8	Execution Time: 0.055 ms	

Рисунок 20 — План выполнения запроса к таблице `rental_rate` после создания составного индекса.

До создания составного индекса для запроса были получены значения Planning Time = 3.787 мс и Execution Time = 0.462 мс. После создания составного индекса по полям `model_id` и `start_date` были получены значения Planning Time = 0.373 мс и Execution Time = 0.055 мс. Если рассматривать суммарное наблюдаемое время работы команды EXPLAIN ANALYZE, то

оно составило 4.249 мс до создания индекса и 0.428 мс после создания индекса. Наблюдается уменьшение как времени построения плана, так и времени выполнения запроса после создания составного индекса. Это можно объяснить тем, что индекс построен по полям, участвующим в фильтрации и сортировке запроса, поэтому для данного запроса его использование оказалось более выгодным.

Выводы

В ходе выполнения лабораторной работы №4 были закреплены практические навыки построения запросов на выборку данных к базе данных PostgreSQL, создания представлений, использования подзапросов при модификации данных и анализа выполнения запросов с помощью индексов и команды EXPLAIN ANALYZE. На основе ранее разработанной базы данных carsharing по предметной области «Прокат автомобилей» были выполнены семь запросов на выборку данных в соответствии с индивидуальным заданием варианта 12, созданы два представления, а также составлены три запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов. Кроме того, был проведён эксперимент с простым и составным индексами, в ходе которого выполнено сравнение планов и времени выполнения запросов до и после создания индексов. Полученные результаты показали, что на небольшом объёме тестовых данных PostgreSQL может сохранять последовательное чтение таблиц, что является допустимым поведением планировщика. Цель лабораторной работы достигнута.